

UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

ICN-292 Sistemas de Información para la Gestión

Segundo Semestre 2026

Laboratorio 5

Profesor:

José Miguel González Paul

Alumno:

Thannia Contreras

Github:

https://github.com/thanniacontreras/ICN292-Lab5-Contreras-Thannia

Desarrollo

1) Consulta 1: cantidad de ventas por vendedor.

Esta consulta permite identificar la cantidad de ventas registradas por cada vendedor de Andes Bike, incluyendo aquellos que no han realizado ninguna operación. Para ello, se utiliza LEFT JOIN, que conserva todos los vendedores, y COUNT, que contabiliza sus ventas. Finalmente, los resultados se ordenan de mayor a menor cantidad, utilizando el nombre como criterio de desempate.

```sql
-- CONSULTA 1
SELECT
    vendedores.nombre AS vendedor,
    COUNT (ventas.id_venta) AS cantidad_ventas
FROM vendedores
LEFT JOIN ventas
    ON vendedores.id_vendedor = ventas.id_vendedor
GROUP BY vendedores.id_vendedor, vendedores.nombre
ORDER BY cantidad_ventas DESC, vendedor ASC;
```

vendedor	cantidad_ventas
Camila Ortiz	4
Diego Fuentes	3
Ignacio Vera	3
Javiera Soto	3
Paula Riquelme	3
Matías Herrera	2
Fernanda Lillo	0
Rubén Castro	0

2) Consulta 2: ventas y unidades vendidas por tienda.

La consulta determina la cantidad total de ventas y unidades comercializadas por cada tienda. Mediante LEFT JOIN, se incluyen los locales sin vendedores o sin ventas registradas. Además, se utilizan COUNT y SUM para calcular los indicadores, mientras que COALESCE permite reemplazar los valores nulos por cero.

```sql
-- CONSULTA 2
SELECT
    tiendas.nombre AS tienda,
    COUNT(ventas.id_venta) AS cantidad_ventas,
    COALESCE(SUM(ventas.unidades), 0) AS unidades_vendidas
FROM tiendas
LEFT JOIN vendedores
    ON tiendas.id_tienda = vendedores.id_tienda
LEFT JOIN ventas
    ON vendedores.id_vendedor = ventas.id_vendedor
GROUP BY tiendas.id_tienda, tiendas.nombre
ORDER BY unidades_vendidas DESC;
```

tienda	cantidad_ventas	unidades_vendidas
Andes Concepción	6	14
Andes Providencia	7	11
Andes Maipú	5	10
Andes La Serena	0	0

3) Consulta 3: tiendas sin vendedores asignados.

Esta consulta identifica los locales que no cuentan con vendedores asignados. Para ello, se utiliza LEFT JOIN para conservar todas las tiendas y la condición IS NULL para seleccionar aquellas que no presentan registros relacionados en la tabla de vendedores.

```sql
-- CONSULTA 3
SELECT
    tiendas.nombre AS tienda
FROM tiendas
LEFT JOIN vendedores
    ON tiendas.id_tienda = vendedores.id_tienda
WHERE vendedores.id_vendedor IS NULL;
```

tienda
Andes La Serena

4) Consulta 4: modelos sin ventas durante junio de 2025.

La consulta identifica los modelos de bicicletas que no registraron ventas durante junio de 2025. Se utiliza LEFT JOIN para conservar el catálogo completo y una condición de fecha para considerar exclusivamente

las ventas del período analizado. Posteriormente, IS NULL permite identificar los modelos sin operaciones.

```sql
-- CONSULTA 4
SELECT
    bicicletas.modelo
FROM bicicletas
LEFT JOIN ventas
    ON bicicletas.id_bicicleta = ventas.id_bicicleta
    AND ventas.fecha >= '2025-06-01'
    AND ventas.fecha < '2025-07-01'
WHERE ventas.id_venta IS NULL
ORDER BY bicicletas.modelo;
```

modelo
Ruta Austral
Sprint R7
Urbana Pro
Volt City

5) Consulta 5: vendedores sin ventas y su tienda

Esta consulta identifica a los vendedores que no han registrado ventas y muestra la tienda a la que pertenecen. Se utilizan dos operaciones LEFT JOIN para mantener todos los vendedores, incluso aquellos sin tienda asignada, mientras que IS NULL permite seleccionar únicamente a quienes no presentan operaciones comerciales.

```sql
-- CONSULTA 5
SELECT
    vendedores.nombre AS vendedor,
    tiendas.nombre AS tienda
FROM vendedores
LEFT JOIN ventas
    ON vendedores.id_vendedor = ventas.id_vendedor
LEFT JOIN tiendas
    ON vendedores.id_tienda = tiendas.id_tienda
WHERE ventas.id_venta IS NULL
ORDER BY vendedores.nombre;
```

	vendedor	tienda
▶	Fernanda Lillo	Andes Providencia
	Rubén Castro	NULL

6) Consulta 6: pares de vendedores que trabajan en la misma tienda.

La consulta identifica los pares de vendedores que comparten un mismo local mediante un SELF JOIN, que permite relacionar la tabla de vendedores consigo misma. Se comparan las fechas de ingreso para posicionar primero al trabajador más antiguo y se utilizan los identificadores como criterio de desempate, evitando que una misma pareja aparezca duplicada.

```sql
-- CONSULTA 6
SELECT
    vendedor1.nombre AS primer_vendedor,
    vendedor1.fecha_ingreso AS ingreso_primero,
    vendedor2.nombre AS segundo_vendedor,
    vendedor2.fecha_ingreso AS ingreso_segundo,
    tiendas.nombre AS tienda
FROM vendedores AS vendedor1
JOIN vendedores AS vendedor2
    ON vendedor1.id_tienda = vendedor2.id_tienda
    AND (
        vendedor1.fecha_ingreso < vendedor2.fecha_ingreso
        OR (
            vendedor1.fecha_ingreso = vendedor2.fecha_ingreso
            AND vendedor1.id_vendedor < vendedor2.id_vendedor
        )
    )
JOIN tiendas
    ON vendedor1.id_tienda = tiendas.id_tienda
ORDER BY
    tiendas.nombre,
    vendedor1.fecha_ingreso,
    vendedor1.nombre;
```

	primer_vendedor	ingreso_primero	segundo_vendedor	ingreso_segundo	tienda
▶	Paula Riquelme	2022-11-02	Ignacio Vera	2024-08-19	Andes Concepción
	Javiera Soto	2023-07-24	Matías Herrera	2024-01-15	Andes Maipú
	Diego Fuentes	2024-01-15	Camila Ortiz	2024-06-20	Andes Providencia
	Diego Fuentes	2024-01-15	Fernanda Lillo	2025-02-03	Andes Providencia
	Camila Ortiz	2024-06-20	Fernanda Lillo	2025-02-03	Andes Providencia

7) *Consulta 7: monto total vendido en bicicletas de Montaña y Ruta*

Esta consulta calcula el monto total vendido por tienda considerando únicamente bicicletas de las categorías Montaña y Ruta. El monto se obtiene multiplicando las unidades vendidas por el precio de cada modelo y sumando los resultados. Posteriormente, mediante HAVING, se seleccionan las tiendas cuyas ventas superan los $2.000.000.

```sql
-- CONSULTA 7
SELECT
    tiendas.nombre AS tienda,
    SUM(ventas.unidades * bicicletas.precio) AS monto_total
FROM tiendas
JOIN vendedores
    ON tiendas.id_tienda = vendedores.id_tienda
JOIN ventas
    ON vendedores.id_vendedor = ventas.id_vendedor
JOIN bicicletas
    ON ventas.id_bicicleta = bicicletas.id_bicicleta
WHERE bicicletas.categoria IN ('Montaña', 'Ruta')
GROUP BY tiendas.id_tienda, tiendas.nombre
HAVING SUM(ventas.unidades * bicicletas.precio) > 2000000
ORDER BY monto_total DESC;
```

tienda	monto_total
Andes Providencia	4399930.00
Andes Concepción	3499950.00

8) *Consulta 8: bicicletas con precio superior al promedio del catálogo.*

La consulta permite identificar los modelos de bicicletas cuyo precio supera el promedio general del catálogo. Para ello, se utiliza una subconsulta con la función AVG, que calcula el precio promedio de los ocho modelos registrados. Posteriormente, se seleccionan únicamente las bicicletas que superan dicho valor y se ordenan de mayor a menor precio.

```sql
-- CONSULTA 8
SELECT
    bicicletas.modelo,
    bicicletas.precio
FROM bicicletas
WHERE bicicletas.precio > (
    SELECT AVG(precio)
    FROM bicicletas
)
ORDER BY bicicletas.precio DESC;
```

modelo	precio
Volt City	1249990.00
Sprint R7	1099990.00
Ruta Austral	899990.00
Trail Master	749990.00

9) *Consulta 9: vendedores con promedio de unidades superior al promedio global.*

Esta consulta identifica a los vendedores cuyo promedio de unidades por venta supera el promedio general de la cadena. Se utiliza AVG para calcular el promedio individual de cada vendedor y una subconsulta para determinar el promedio global. Finalmente, mediante HAVING, se seleccionan únicamente los vendedores cuyo desempeño supera dicho indicador.

```sql
-- CONSULTA 9
SELECT
    vendedores.nombre AS vendedor,
    AVG(ventas.unidades) AS promedio_unidades
FROM vendedores
JOIN ventas
    ON vendedores.id_vendedor = ventas.id_vendedor
GROUP BY vendedores.id_vendedor, vendedores.nombre
HAVING AVG(ventas.unidades) > (
    SELECT AVG(unidades)
    FROM ventas
)
ORDER BY promedio_unidades DESC, vendedores.nombre ASC;
```

	vendedor	promedio_unidades
▶	Paula Riquelme	2.6667
	Javiera Soto	2.3333
	Diego Fuentes	2.0000
	Ignacio Vera	2.0000

10) *Consulta 10: monto total vendido por vendedor.*

Esta consulta calcula el monto total de ventas generado por cada vendedor, incluyendo aquellos que no registraron operaciones. Para ello, se utiliza LEFT JOIN para conservar todos los vendedores y SUM para calcular los ingresos obtenidos, multiplicando las unidades vendidas por el precio de cada bicicleta. Los resultados se presentan de mayor a menor monto total.

```sql
-- CONSULTA 10
SELECT
    vendedores.nombre AS vendedor,
    SUM(ventas.unidades * bicicletas.precio) AS monto_total
FROM vendedores
LEFT JOIN ventas
    ON vendedores.id_vendedor = ventas.id_vendedor
LEFT JOIN bicicletas
    ON ventas.id_bicicleta = bicicletas.id_bicicleta
GROUP BY vendedores.id_vendedor, vendedores.nombre
ORDER BY monto_total DESC, vendedores.nombre ASC;
```

	vendedor	monto_total
▶	Camila Ortiz	3429950.00
	Paula Riquelme	3149920.00
	Matías Herrera	3049970.00
	Diego Fuentes	3009940.00
	Ignacio Vera	2319940.00
	Javiera Soto	2009930.00
	Fernanda Lillo	NULL
	Rubén Castro	NULL

11) *Modelo entidad-relación.*

